Bài tập về Java và Hướng đối tượng

Lưu ý: Sinh viên cần hoàn thiện các Bài 1.1-1.10 trong tuần 1. Các bài còn lại hoàn thiện trong tuần 2.

Bài 1.1: Hello World

Yêu cầu:

1. Tải và cài đặt Oracle JDK tại đây. Lưu ý, ở đây sử dụng Oracle JDK.

2. Thêm Java vào biến môi trường Path.
   - Bước 1: Sử dụng Windows Search (`Windows + S`) tìm kiếm "Edit the system environment variable". Trong hộp thoại "System Properties" hiện lên, chọn "Environment Variables".
   - Bước 2: Trong hộp thoại Environment Variables, vào phần System variables và chọn New.
   - Bước 3: Trong hộp thoại New System Variable, điền vào ô Variable name là JAVA_HOME, còn trong ô Variable value nhấn vào nút Browse Directory và trỏ tới đường dẫn cài đặt JDK. Nhấn OK để hoàn tất việc đặt tên biến môi trường.
   - Bước 4: Tiếp theo, cũng trong phần System variables, tiến hành sửa đổi biến môi trường Path như sau: Kích chuột vào dòng Path và chọn Edit. Hộp thoại Edit environment variable xuất hiện, chúng ta nhấn chuột vào nút New và điền vào dòng sau: %JAVA_HOME%\bin, nhấn OK để kết thúc.
   - Bước 5: Kiểm tra bằng cách mở cmd, nhập vào dòng: java –version. Nếu thông tin hiển thị ra là version Java đã cài đặt là thành công.

3. Viết chương trình `HelloName.java` in ra "Hello, [Tên của bạn]".

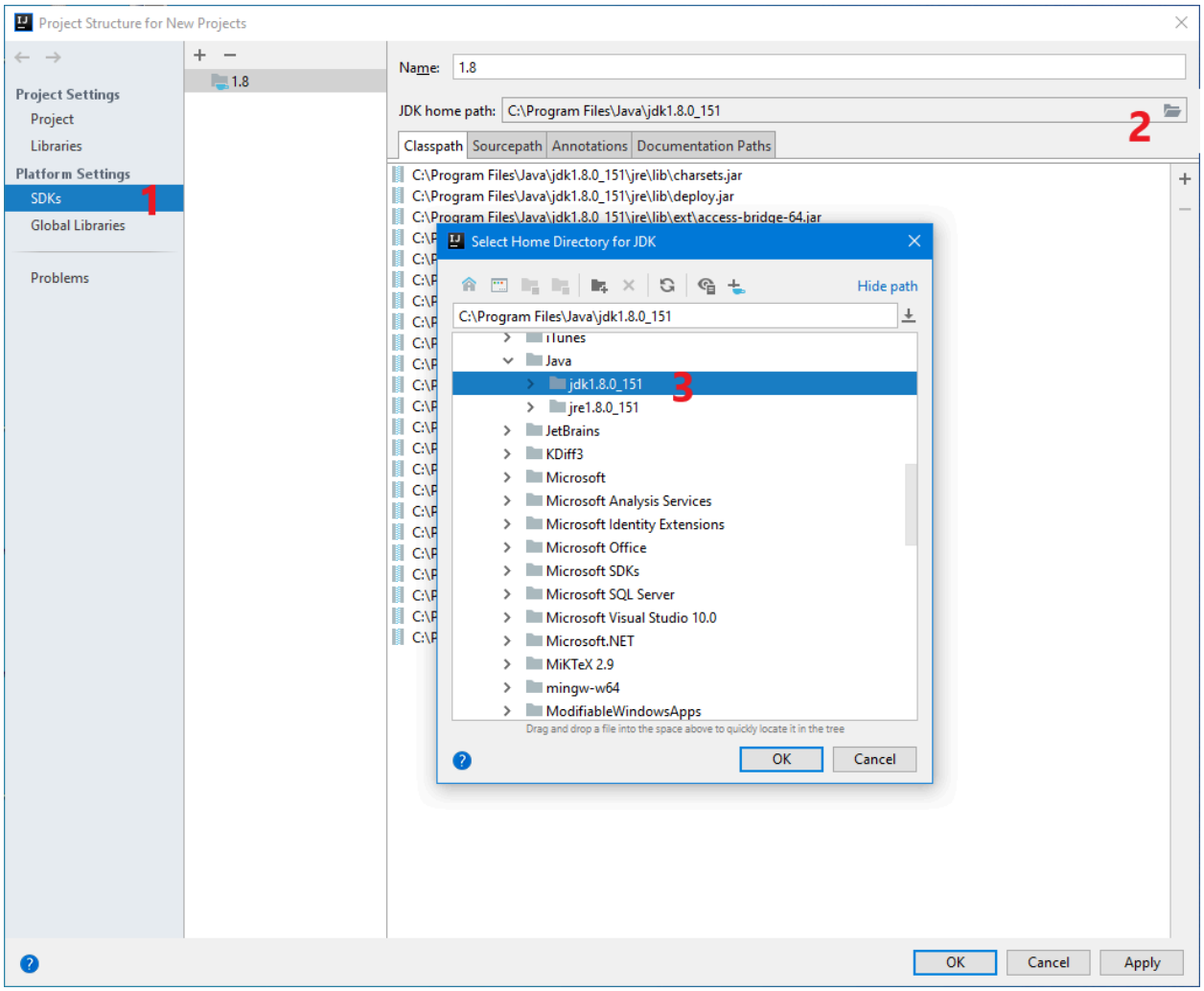
4. **Bắt buộc**: Không dùng IDE (IntelliJ/Eclipse), phải viết bằng Notepad/TextEditor hoặc các trình soạn thảo khác, biên dịch bằng `javac` và chạy bằng `java` trong Terminal/CMD.

Bài 1.2: Cài đặt IDE

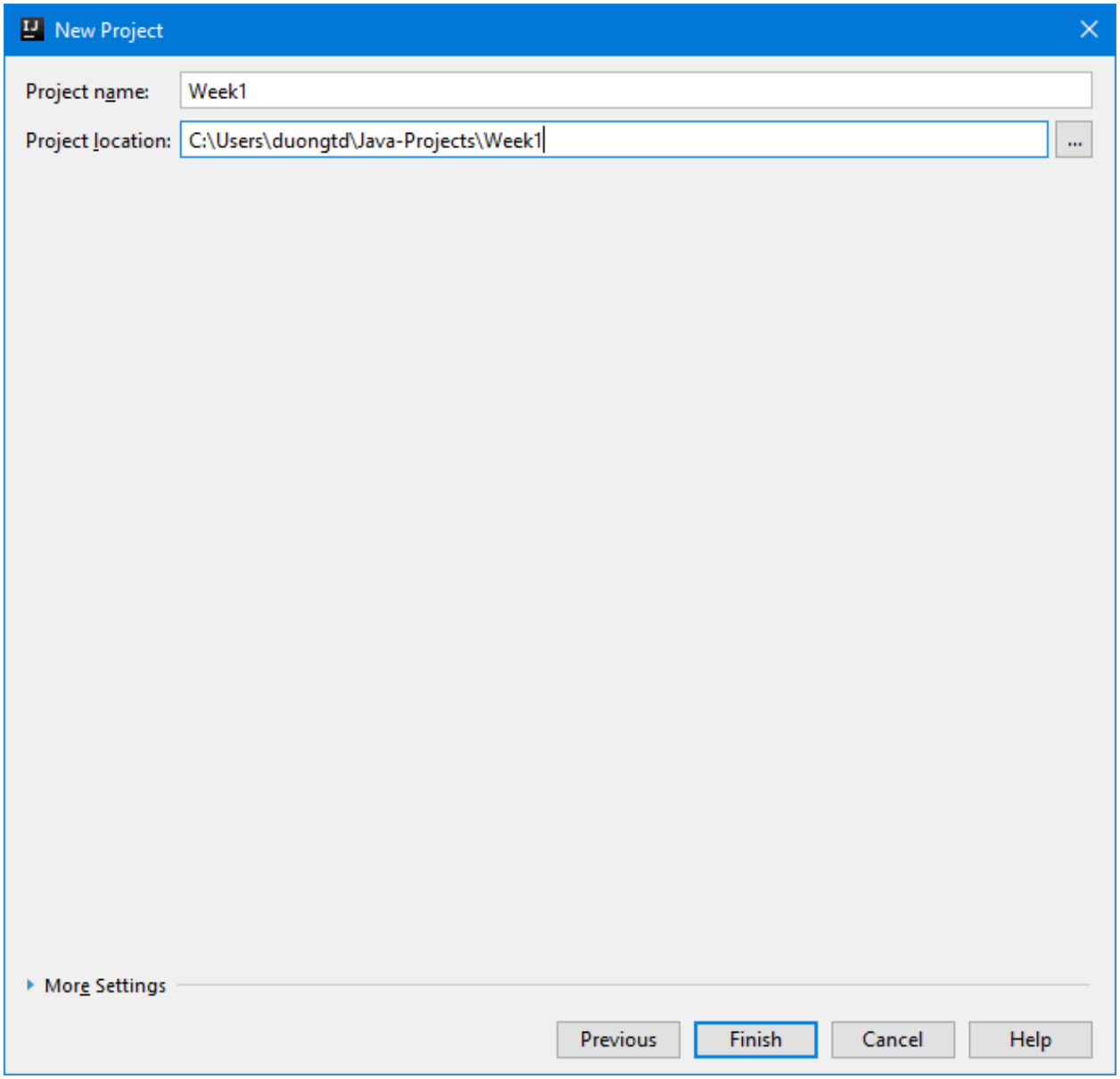
Tải và cài đặt Intellij. Sau đó config đường dẫn tới JDK đã cài đặt:

1. Click *Configure -> Project Defaults -> Project Structure*

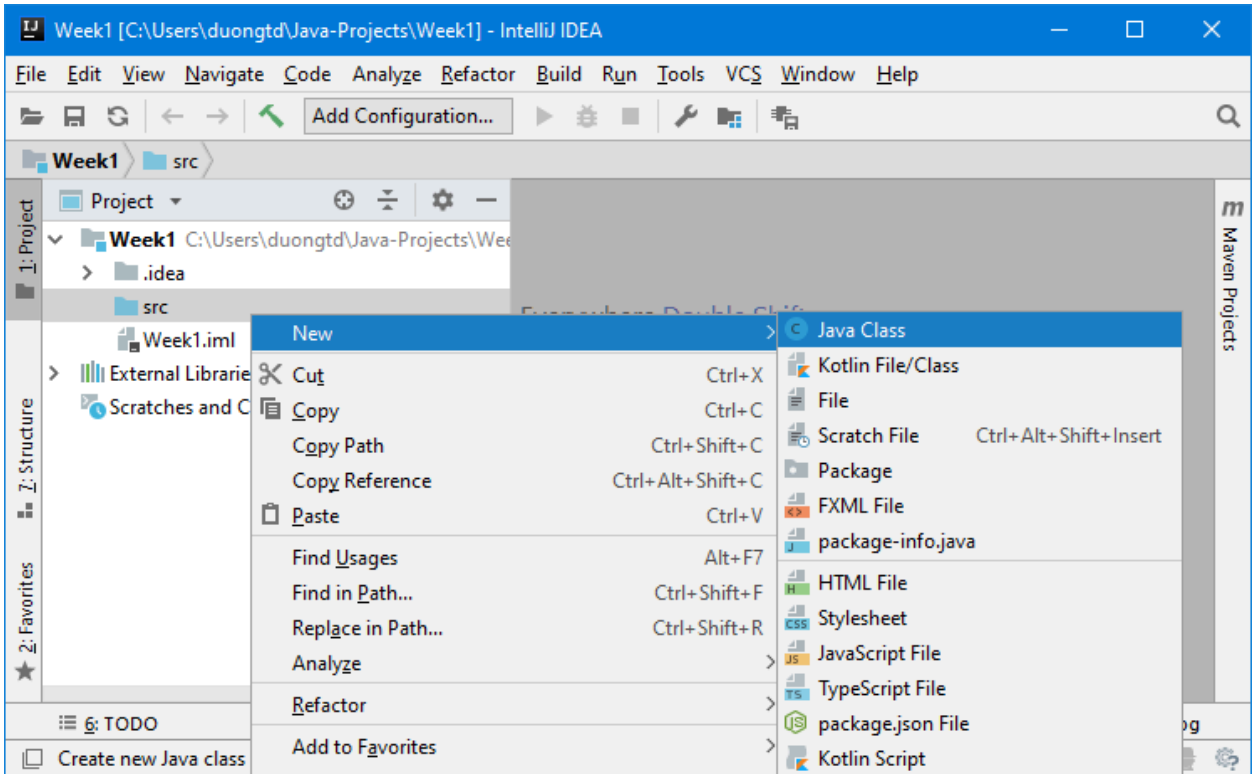
2. Cửa sổ mới hiện lên, ở cột bên trái chọn SDKs; bên phải click để chọn đường dẫn tới JDK home path. Sau khi chọn thành công click OK để ghi nhận.



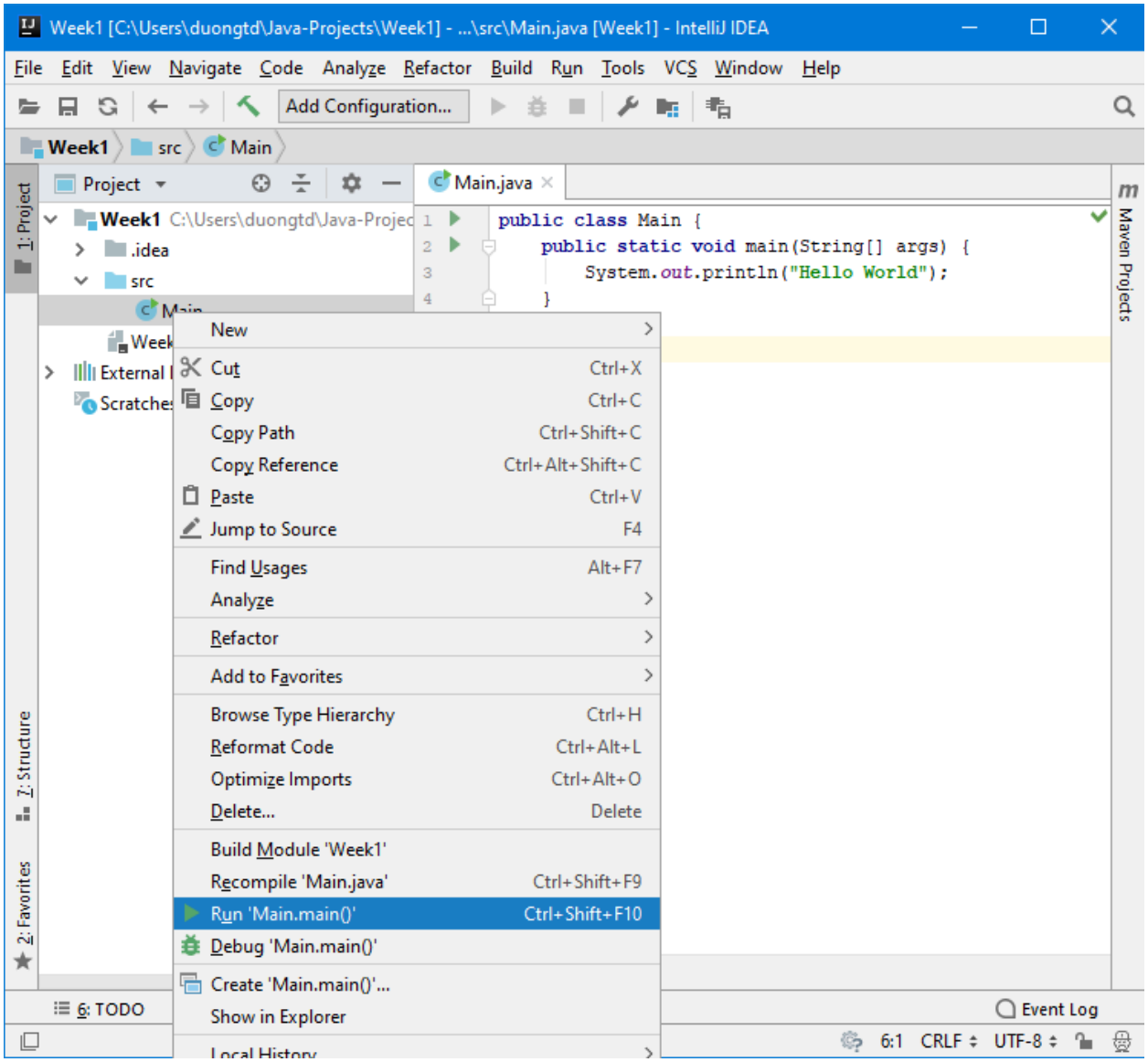
3. Tiếp theo, để tạo mới project, click vào Create New Project ở cửa sổ đầu tiên, click Next, Next (2 lần). Ở cửa sổ cuối cùng, yêu cầu điền tên project và nơi lưu mã nguồn tương ứng. Sau khi hoàn thành, click Finish để xác nhận tạo mới project.



4. Tạo mới lớp Student bằng cách: right-click src -> New -> Java Class; sau đó đặt tên cho class là Student và click OK. Tự hoàn thiện các yêu cầu còn lại.



5. Để chạy chương trình, right-click class chứa phương thức main cần chạy, chọn Run (hoặc Debug).



Bài 1.3: Phân tích Object

Hãy chỉ ra đâu là trạng thái, hành vi, định danh của đối tượng trong đoạn mã dưới đây:

a) BankAccount

```
public class BankAccount {
    private String accountNumber;
    private String ownerName;
    private double balance = 0.0;

    public void openAccount(String accNum, String owner) {
```

```
        this.accountNumber = accNum;
        this.ownerName = owner;
    }

    public void deposit(double amount) {
        this.balance += amount;
    }

    public boolean withdraw(double amount) {
        if (this.balance >= amount) {
            this.balance -= amount;
            return true;
        }
        return false;
    }

    public static void main(String[] args) {
        BankAccount myAccount = new BankAccount();
        myAccount.openAccount("101202303", "Nguyen Van A");
        myAccount.deposit(500.0);
        myAccount.withdraw(150.0);
    }
}
```

b) SmartFan

```
public class SmartFan {
    private String brand = "Xiaomi";
    private boolean isPowerOn = false;
    private int speedLevel = 0; // Mức từ 1 đến 3

    public void turnOn() {
        this.isPowerOn = true;
    }

    public void turnOff() {
        this.isPowerOn = false;
        this.speedLevel = 0;
    }

    public void setSpeed(int newSpeed) {
```

```
        // Chỉ đổi tốc độ nếu quạt đang bật
        if (this.isPowerOn == true) {
            this.speedLevel = newSpeed;
        }
    }

    public static void main(String[] args) {
        SmartFan livingRoomFan = new SmartFan();
        livingRoomFan.turnOn();
        livingRoomFan.setSpeed(2);

        SmartFan bedroomFan = new SmartFan();
        bedroomFan.setSpeed(3);
        bedroomFan.turnOn();
    }
}
```

Bài 1.4: Fibonacci

Viết các phương thức sau, sử dụng code mẫu tại:

https://github.com/oasis-homework/Week3/blob/main/Solution.java

Yêu cầu: Tạo lớp Solution với các đặc điểm sau:

1. **Phương thức (Methods):**
   - fibonacci(long n): Để tính số thứ n trong dãy Fibonacci. Phương thức có kiểu trả về là long, mức truy nhập public.
2. **Yêu cầu:** Triển khai hàm fibonnacci theo công thức như sau:

$$F_n := F(n) := \begin{cases} 0\,, & \text{khi } n = 0\,; \\ 1, & \text{khi } n = 1; \\ F(n-1) + F(n-2) & \text{khi } n > 1. \end{cases}$$

   Trường hợp n < 0, hàm trả về giá trị -1 Trường hợp số Fibonacci thứ n vượt quá giá trị lớn nhất của kiểu dữ liệu long, phương thức trả về giá trị Long.MAX_VALUE.

3. **Điều kiện:** n là số tự nhiên, n <=100.

4. **Hàm main:**
    - Viết hàm main tạo đối tượng Solution, gọi đến hàm fibonacci thử nghiệm với nhiều giá trị n khác nhau.

Bài 1.5: Xây dựng ước số chung lớn nhất

Yêu cầu: Tạo lớp Solution với các đặc điểm sau:

1. **Phương thức (Methods):**
    - gcd (int a, int b): Để tìm ước số chung lớn nhất của 2 số nguyên a và b. Kiểu trả về int.
2. **Yêu cầu:**
    - Triển khai hàm gcd.
    - Viết hàm main tạo đối tượng solution, gọi đến hàm gcd. Thử nghiệm các giá trị khác nhau (chú ý những giá trị đặc biệt).

Bài 1.6: Kiểm tra số nguyên tố

Yêu cầu: Tạo lớp Solution với các đặc điểm sau:

1. **Phương thức (Methods):**
    - isPrime(int n): Để kiểm tra xem một số nguyên n có phải là một số nguyên tố hay không. Kiểu trả về boolean.
2. **Yêu cầu:**
    - Triển khai phương thức isPrime.
    - Viết hàm main tạo đối tượng Solution, gọi đến hàm isPrime. Thử với các giá trị n khác nhau (số âm, số rất lớn, …) và in ra kết quả.
    - Không dùng thư viện có sẵn.
    - Độ phức tạp tối ưu hơn O(n).

Bài 1.7: Đảo ngược số nguyên

Yêu cầu: Tạo lớp Solution với các đặc điểm sau:

1. **Phương thức (Methods):**
    - reverse(int n): Để đảo ngược thứ tự các chữ số của một số nguyên n. Kiểu trả về int.
2. **Yêu cầu:**
    - Triển khai phương thức reverse.

- Viết hàm main tạo đối tượng Solution, gọi đến hàm reverse. Thử với các giá trị n khác nhau (số âm, số có chữ số 0 ở cuối, số lớn, …) và in ra kết quả.
- Không sử dụng thư viện có sẵn để đảo ngược số.
- Nếu kết quả vượt quá phạm vi của kiểu dữ liệu int, phương thức trả về giá trị 0.

Bài 1.8: Kiểm tra số Palindrome

Yêu cầu: Tạo lớp Solution với các đặc điểm sau:

1. **Phương thức (Methods):**
   - isPalindrome(int n): Để kiểm tra xem một số nguyên n có phải là số Palindrome hay không. Kiểu trả về boolean.

     (Một số được gọi là Palindrome nếu giá trị của nó không thay đổi khi đảo ngược thứ tự các chữ số)

2. **Yêu cầu:**
   - Triển khai phương thức isPalindrome.
   - Viết hàm main tạo đối tượng Solution, gọi đến hàm isPalindrome. Thử với các giá trị n khác nhau (số âm, số có chữ số 0 ở cuối, số lớn, …).
   - Không sử dụng thư viện có sẵn để kiểm tra.

Bài 1.9: Tính tổng chữ số

Yêu cầu: Tạo lớp Solution với các đặc điểm sau:

1. **Phương thức (Methods):**
   - sumOfDigits(int n): Để tính tổng các chữ số của một số nguyên n. Kiểu trả về int.
2. **Yêu cầu:**
   - Triển khai phương thức sumOfDigits.
   - Viết hàm main tạo đối tượng Solution, gọi đến hàm sumOfDigits. Thử với các giá trị n khác nhau (số âm, số có nhiều chữ số, số bằng 0, …).
   - Không sử dụng thư viện có sẵn.

Bài 1.10: Tìm số lớn thứ 2 trong mảng

Yêu cầu: Tạo lớp Solution với các đặc điểm sau:

1. **Phương thức (Methods):**
   - secondLargest(int[] arr): Để tìm số lớn thứ hai trong một mảng số nguyên arr. Kiểu trả về int.
2. **Yêu cầu:**
   - Triển khai phương thức secondLargest.
   - Viết hàm main tạo đối tượng Solution, gọi đến hàm secondLargest. Thử với các mảng có giá trị khác nhau (mảng bình thường, mảng có phần tử trùng nhau, mảng có 1 phần tử, …).

     (Nếu mảng không có phần tử lớn thứ 2, trả về giá trị -1. Số lớn thứ hai phải khác số lớn nhất)

   - Không sử dụng thư viện có sẵn để sắp xếp mảng (ví dụ: Arrays.sort).
   - Độ phức tạp yêu cầu O(n).

Bài 2.1: Bank Account

Mô phỏng một tài khoản ngân hàng đơn giản

Yêu cầu: Tạo lớp BankAccount với các đặc điểm sau:

1. **Thuộc tính (Fields):**
   - accountNumber (String): Chỉ đọc, không thay đổi sau khi tạo.
   - balance (double): Số dư, không được phép truy cập trực tiếp từ bên ngoài.
   - ownerName (String): Tên chủ tài khoản.
2. **Hàm khởi tạo (Constructors):**
   - Constructor nhận vào accountNumber và ownerName. Số dư mặc định là 0.
   - Constructor nhận đủ 3 tham số. Nếu số dư truyền vào < 0, gán mặc định bằng 0 và in thông báo lỗi.
3. **Phương thức (Methods):**
   - deposit(double amount): Nạp tiền. Số tiền nạp phải > 0.
   - withdraw(double amount): Rút tiền. Số tiền rút phải > 0 và <= số dư hiện tại. Trả về true nếu rút thành công, false nếu thất bại.
   - getBalance(): Chỉ cho phép xem số dư, không cho phép sửa.
4. **Hàm Main:**
   - Viết hàm Main theo các kịch bản: nạp tiền âm, rút quá số dư, rút tiền hợp lệ để chứng minh logic bảo vệ dữ liệu hoạt động đúng.

Bài 2.2: Student

Yêu cầu: Xây dựng lớp Student mô tả sinh viên với các thuộc tính: id (mã SV), name (tên), email, gpa (điểm trung bình).

1. **Encapsulation:** Tất cả các thuộc tính phải là private.
2. **Constructor:** Tạo 3 constructor:
    - Không tham số.
    - Có tham số (id, name).
    - Có đầy đủ tham số (Copy constructor).
3. **Validation:** Trong các hàm setter (ví dụ: setGpa), phải kiểm tra logic (VD: GPA phải từ 0.0 đến 4.0). Nếu sai, phải in ra lỗi hoặc giữ nguyên giá trị cũ.
4. Viết hàm main tạo 3 sinh viên bằng 3 cách khác nhau, thử gán GPA < 0 và in thông tin ra màn hình.

Bài 2.3: The swap trick

Yêu cầu:

1. Tạo class NumberWrapper chứa một số nguyên int value. Tạo setter/getter tương ứng.
2. Viết hàm swap(NumberWrapper a, NumberWrapper b) thực hiện hoán đổi vị trí 2 tham số a và b (gán temp = a; a = b; b = temp;).
3. Viết hàm main:
    - Tạo 2 đối tượng n1 (value=5) và n2 (value=10).
    - Gọi swap(n1, n2).
    - In giá trị của n1, n2 sau khi gọi hàm.

Bài 2.4: Composition & copy

Yêu cầu:

1. Tạo class MyDate (day, month, year).
2. Tạo class Employee gồm: name (String) và birthday (kiểu MyDate).
3. Viết **Copy Constructor** cho Employee (Constructor nhận vào một Employee khác để sao chép).
4. Trong main:
    - Tạo emp1 với ngày sinh 1/1/2000.
    - Tạo emp2 bằng cách sao chép emp1: Employee emp2 = new Employee(emp1);
    - Thay đổi ngày sinh của emp1 thành 2/2/2022.

    - In ngày sinh của emp2.
5. **Yêu cầu quan trọng:** Đảm bảo khi sửa emp1, emp2 **KHÔNG** bị thay đổi theo. (Thực hiện Deep Copy thay vì Shallow Copy).

Bài 2.5: Equals Method

Yêu cầu: Tạo lớp Book với các đặc điểm sau:

1. **Thuộc tính (Fields):** title (String), author (String), price (double). Tất cả thuộc tính là private.
2. **Hàm khởi tạo (Constructors):** Constructor nhận đủ 3 tham số.
3. **Phương thức (Methods):**
    - Override phương thức equals(Object obj)
    - Equals trả về true nếu title, author, price giống nhau.
4. **Hàm Main:**
    - Tạo 2 object có cùng dữ liệu
    - So sánh bằng == và equals
    - In kết quả
5. **Câu hỏi:** == và equals khác nhau như thế nào?

Bài 2.6: Immutable Object & Deep Copy

Mục tiêu: Hiểu sâu về Encapsulation (Đóng gói), tham chiếu bộ nhớ và bảo mật dữ liệu.

Mô tả nghiệp vụ: Trong hệ thống ngân hàng, một Transaction sau khi được tạo ra thì tuyệt đối không được phép chỉnh sửa. Một Account sẽ lưu trữ một mảng các giao dịch này.

Yêu cầu thiết kế & Cài đặt:

1. **Lớp Transaction**: gồm transactionId, amount, timestamp (dùng String đơn giản).
    - Đảm bảo không ai có thể sửa đổi dữ liệu của Transaction sau khi gọi constructor.
2. **Lớp Account**: gồm accountId, balance và mảng Transaction[] history.
    - Có phương thức addTransaction(Transaction t).
    - Có phương thức Transaction[] getHistory().
3. **Trong hàm main:**
    - Viết code đóng vai một hacker: Lấy mảng lịch sử giao dịch từ hàm getHistory(), sau đó cố tình sửa giá trị amount của một giao dịch trong mảng đó thành 9999999. Hoặc gọi history[0] = null.
    - **Yêu cầu bắt buộc:** Hãy thiết kế lớp Account sao cho code của "hacker" dù chạy thành công nhưng dữ liệu gốc bên trong đối tượng Account vẫn không bị thay đổi.

Bài 2.7: Pass-by-Reference & Deep Copy

Mục tiêu: Phân biệt "Shallow Copy" (Sao chép nông) và "Deep Copy" (Sao chép sâu).

Mô tả nghiệp vụ: Hệ thống bán hàng có lớp Product và lớp Inventory (Kho hàng). Kho hàng quản lý một mảng các sản phẩm.

Yêu cầu thiết kế & Cài đặt:

1. Tạo lớp Product: gồm id, name, price.
2. Tạo lớp Inventory:
    - Thuộc tính Product[] items.
    - Hàm khởi tạo: public Inventory(Product[] initialItems) gán mảng đưa vào thành mảng của kho.
3. **Trong hàm main:**
    - Tạo một mảng Product[] arr gồm 2 sản phẩm (Vd: Laptop giá 1000$).
    - Khởi tạo kho: Inventory kho = new Inventory(arr).

- Bên ngoài hàm main, gọi arr[0].setPrice(5000$).
- Sau đó, in danh sách sản phẩm trong đối tượng kho ra màn hình.

Bài 2.8: Object Reference và Garbage Collection

Yêu cầu: Tạo lớp Person với các đặc điểm sau:

1. **Thuộc tính (Fields):**
   - name(String): Tên của đối tượng.
   - me(Person): Tham chiếu đến một đối tượng Person khác.
   - Tất cả thuộc tính khai báo là private.
2. **Hàm khởi tạo (Constructors):** Constructor nhận vào tham số name và gán cho thuộc tính tương ứng.
3. **Phương thức (Methods):**
   - void setMe(Person other): Gán đối tượng other cho tham chiếu me.
   - Person getMe(): Trả về đối tượng mà me đang tham chiếu.
   - String getName(): Trả về name
4. **Hàm main:**
   - Khởi tạo một đối tượng Person và gán cho biến p.
   - Set tham chiếu me đến đối tượng mà biến p đang tham chiếu.
   - Truy cập và in kết quả của phương thức getName() thông qua biến tham chiếu me của đối tượng p.
   - Set biến p = null.
5. **Yêu cầu:**
   - Sau khi setMe(p) có bao nhiêu đối tượng Person tồn tại trong bộ nhớ?
   - Sau dòng lệnh p = null; đối tượng Person có bị xóa ngay lập tức khỏi bộ nhớ không? Giải thích cơ chế hoạt động của Garbage Collection trong trường hợp này.
   - Đối tượng Person có thể được truy cập lại không? Giải thích.
   - Vẽ sơ đồ bộ nhớ (Stack và Heap) tại 2 thời điểm: trước và sau khi p = null.

Bài 2.9: Static Methods and Standard IO:

Yêu cầu: Bạn đang xây dựng phần mềm quản lý kho cho chuỗi cửa hàng. Hệ thống cần quản lý các sản phẩm riêng lẻ, nhưng cũng phải theo dõi các thông số chung của toàn hệ thống như mức thuế VAT và tổng doanh thu toàn chuỗi. Tạo lớp Product với các đặc điểm sau:

1. **Thuộc tính (Fields):** Tất cả các thuộc tính được khai báo là **private**.

- **Instance Fields:** name (String), price (double), quantity (int) - số lượng hàng tồn kho và discount (double) - Mức giảm giá cho từng sản phẩm.
- **Static Fields:** taxRate (double) - Mức thuế VAT chung (mặc định là 0.1) và totalRevenue (double) - Tổng doanh thu từ tất cả các giao dịch bán hàng (khởi tạo bằng 0).

2. **Constructor:** Constructor nhận đủ các tham số để gán giá trị cho (name, price, quantity, và discount).
3. **Phương thức (Methods):**
   - static void updateTaxRate(double newRate): Cập nhật mức thuế VAT mới cho toàn hệ thống.
   - double calculateFinalPrice(): Tính giá cuối cùng của sản phẩm sau khi trừ giảm giá riêng và cộng thuế VAT dùng chung. Công thức: finalPrice = (price - discount) x (1 + taxRate)
   - void updateDiscount(double newDiscount): Cập nhật mức giảm giá mới cho sản phẩm.
   - void sell(int amount): Nếu amount <= quantity: Trừ số lượng tồn kho, tính tiền thực tế thu được (amount x finalPrice) và cộng dồn vào biến tĩnh totalRevenue. In thông báo thành công ra **System.out**. Nếu amount > quantity: In thông báo lỗi "Không đủ hàng trong kho" ra **System.err.**
4. **Hàm main:**
   - Sử dụng lớp Scanner: Nhập liệu dữ liệu và khởi tạo 2 đối tượng Product p1 và p2 với thông tin (name, price, quantity, discount) nhập từ bàn phím.
   - Thực hiện giao dịch: Yêu cầu người dùng nhập số lượng cần mua cho p1 và p2 và gọi hàm p1.sell() và p2.sell().
   - Kiểm tra tính chất Static:
     i. In giá cuối cùng (calculateFinalPrice) của cả p1 và p2.
     ii. Gọi hàm tĩnh Product.updateTaxRate(0.08) (giảm thuế xuống 8%). In lại giá cuối cùng của p1 và p2 một lần nữa để quan sát sự thay đổi trong mức giá của cả hai sản phẩm sau khi giảm thuế.
     iii. Gọi hàm p1.updateDiscount(10.0) (Thay mức giảm của sản phẩm bằng 10). In ra giá trị cuối cùng của p1 và p2 một lần nữa để quan sát sự thay đổi khi chỉ giảm giá của một sản phẩm.
   - In tổng doanh thu toàn hệ thống (totalRevenue).

Bài 2.10: Overloading and ‘this’

Yêu cầu: Bạn đang viết phần mềm điều khiển cho bóng đèn thông minh (SmartLight). Bóng đèn này có các cấp độ sáng khác nhau và có thể được thiết lập nhanh qua các chế độ (Preset). Tạo lớp SmartLight.

1. **Thuộc tính (Fields):**

- id (String): Id của đèn
- name (String): Tên của đèn
- brightness (int): Độ sáng của đèn
- Tất cả thuộc tính khai báo là private.

2. **Hàm khởi tạo (Constructors):** Constructor 1 nhận đủ 3 tham số (id, name và brightness), trong khi Constructor 2 còn lại chỉ sử dụng 2 tham số (id, name) và sử dụng **this(...)** để gọi Constructor 1 và gán brightness mặc định là 50.
3. **Phương thức (Methods):**
   - void setBrightness(int brightness): Cập nhật độ sáng mới (sử dụng this).
   - void setBrightness(String preset)**:** Nạp chồng phương thức trên.
     i. Nếu preset là "MAX", hãy gọi this.setBrightness(100).
     ii. Nếu preset là "MIN", hãy gọi this.setBrightness(10).
     iii. Nếu preset là "ECO", hãy gọi this.setBrightness(30).
   - void connectToHub(CentralHub hub): Gọi phương thức hub.registerDevice(this) để đăng ký thiết bị hiện tại vào bộ điều khiển trung tâm (truyền this làm tham số).
4. **Hàm main:**
   - Khởi tạo một đối tượng hub từ lớp CentralHub.
   - Tạo bóng đèn l1 bằng Constructor đầy đủ (ví dụ: "L01", "Đèn phòng khách", 80).
   - Tạo bóng đèn l2 bằng Constructor 2 tham số (ví dụ: "L02", "Đèn ngủ").
   - Thực hiện lệnh l2.setBrightness("ECO").
   - Gọi l1.connectToHub(hub) và l2.connectToHub(hub).
   - In ra độ sáng hiện tại của cả 2 đèn để kiểm tra kết quả.

Tạo lớp CentralHub:Một class với phương thức duy nhất là registerDevice(SmartLight light), in ra thông báo: "[HUB] Đang kết nối với thiết bị: " + light.getName()